

Métropole – 2024 – sujet2 - Correction

Exercice 2 (6 points)

Partie A – Quelques outils

1. Constructeur de la classe `Marchandise`

```
class Marchandise:
    def __init__(self, p: int, v: int) -> 'Marchandise':
        assert v > 0
        self.prix = p
        self.volume = v
```

👉 *Commentaire : l'assertion garantit que le volume est strictement positif ; sinon une exception est levée.*

2. Création de `m1` (7 L, 20 €)

```
m1 = Marchandise(20, 7)
```

👉 *Commentaire : l'ordre des paramètres est (prix, volume).*

3. Méthode `ratio`

```
def ratio(self) -> float:
    return self.prix / self.volume
```

👉 *Commentaire : le ratio correspond au prix volumique (€/litre).*

4. Fonction `prixListe`

```
def prixListe(tab: list) -> int:
    total = 0
    for m in tab:
        total += m.prix
    return total
```

👉 *Commentaire : on additionne les prix de chaque marchandise du tableau.*

Partie B – Première approche

Marchandises :

- $m1 = (40, 20)$
- $m2 = (210, 70)$
- $m3 = (160, 40)$
- $m4 = (50, 50)$

Capacité : 100 L

5. Combinaisons possibles (≤ 100 L)

Volumes et prix :

- $m1 \rightarrow 20 \text{ L} \rightarrow 40 \text{ €}$
- $m2 \rightarrow 70 \text{ L} \rightarrow 210 \text{ €}$
- $m3 \rightarrow 40 \text{ L} \rightarrow 160 \text{ €}$
- $m4 \rightarrow 50 \text{ L} \rightarrow 50 \text{ €}$

Combinaisons :

- $m1 + m2 \rightarrow 90 \text{ L} \rightarrow 250 \text{ €}$
- $m1 + m3 \rightarrow 60 \text{ L} \rightarrow 200 \text{ €}$
- $m1 + m4 \rightarrow 70 \text{ L} \rightarrow 90 \text{ €}$
- $m2 + m3 \rightarrow 110 \text{ L} \rightarrow \text{impossible}$
- $m2 + m4 \rightarrow 120 \text{ L} \rightarrow \text{impossible}$
- $m3 + m4 \rightarrow 90 \text{ L} \rightarrow 210 \text{ €}$
- $m1 + m3 + m4 \rightarrow 110 \text{ L} \rightarrow \text{impossible}$
- $m1 + m2 + m3 \rightarrow 130 \text{ L} \rightarrow \text{impossible}$

Meilleure solution :

$m1 + m2 \rightarrow \mathbf{250 \text{ €}}$

👉 *Commentaire : l'algorithme glouton ne trouve pas toujours cette solution optimale.*

6. Qualificatif de l'algorithme

Glouton

👉 *Commentaire : il choisit localement la meilleure marchandise (meilleur ratio) sans garantie d'optimalité globale.*

7. Fonction tri

Tri décroissant selon le ratio (tri par insertion par exemple) :

```
def tri(tab: list) -> None:
    for i in range(1, len(tab)):
        cle = tab[i]
        j = i - 1
        while j >= 0 and tab[j].ratio() < cle.ratio():
            tab[j + 1] = tab[j]
            j -= 1
        tab[j + 1] = cle
```

👉 *Commentaire : on décale les éléments tant que leur ratio est inférieur.*

8. Nom et coût du tri

Nom : **tri par insertion**

Coût (pire cas) : **quadratique** car présence d'une double boucle.

👉 *Commentaire : coût en $O(n^2)$.*

9. Fonction charge

```
def charge(tab: list, v_restant: int) -> list:
    tri(tab)
    chargement = []
    for m in tab:
        if m.volume <= v_restant:
            chargement.append(m)
            v_restant -= m.volume
    return chargement
```

👉 *Commentaire : on parcourt la liste triée et on ajoute tant que le volume le permet.*

Partie C – Version récursive optimale

10. Fonction chargeOptimale

```
def chargeOptimale(tab: list, v_restant: int, i: int) -> list:
    n = len(tab)
    if i >= n:
        return []
    if tab[i].volume > v_restant:
        return chargeOptimale(tab, v_restant, i + 1)
    else:
        # Option 1 : on prend la marchandise
        avec = [tab[i]] + chargeOptimale(tab, v_restant - tab[i].volume, i + 1)
        # Option 2 : on ne la prend pas
        sans = chargeOptimale(tab, v_restant, i + 1)
        if prixListe(avec) > prixListe(sans):
            return avec
        else:
            return sans
```

👉 *Commentaire : on explore récursivement les deux possibilités et on garde celle qui maximise le prix.*

Remarque:

Cet algorithme :

- explore toutes les combinaisons possibles
- garantit l'optimalité
- a un coût exponentiel

On reconnaît un problème de type sac à dos (knapsack) résolu ici par exploration exhaustive et récursive.